Instant Traveling Companion Discovery Based on Traffic-Monitoring Streaming Data

Xiongbin Wang1,2, Chen Liu1,2, Meiling Zhu1,2,3

1 Beijing Key Laboratory on Integration and Analysis of Large-scale Stream Data, North China University of Technology, Beijing, China

2 Cloud Computing Research Center, North China University of Technology, Beijing, China

3 School of Computer Science and Technology, Tianjin University, Tianjin, China

suifengzhe001@sina.com, liuchen@ncut.edu.cn, meilingzhu2006@126.com

***Abstract*—Traveling companions are object groups that move together in a period of time. To quickly identify traveling companions from a special kind of streaming traffic data, called Automatic Number Plate Recognition (ANPR) data, this paper proposes a framework and several algorithms to discover companion vehicles. Compared to related approaches, our main contribution is that the framework can instantly detect suspicious companion vehicles with their probabilities when they pass through monitoring cameras. Our framework can be used in many time-sensitive scenarios like taking surveillance on the suspect trackers for specific vehicles. Experiments show that our approach can process streaming ANPR data directly and discover the companion vehicles in nearly real time.**

Keywords—component; traveling companion, ANPR data, instant manner, moment companion

I. INTRODUCTION

Recent advances in positioning technology generate huge volumes of moving object data. Finding movement pattern from such data attracts many research studies [1-6]. A useful one is to find traveling companions which was initially defined in [4]. Generally speaking, a traveling companion represents a group of moving objects that move within arbitrary shape of clusters at certain timestamps [3]. Such techniques have wide applications in various areas like transportation management and military surveillance.

For some special applications, discovery of traveling companions are time-sensitive. For example, to ensure the security, we need to take surveillance on the companion vehicles of taxies or bank cash carriers. These vehicles are apt to be tracked, forced to stop and robbed. To prevent the crimes, suspicious trackers should be identified and reported as soon as possible while the trajectory data stream is arriving. However, most of current researches are designed to work on static datasets on 2D Euclidean space. They still cannot produce their results instantly and directly from real time monitoring data [5].

Nowadays, lots of researches are designed to report traveling companions based on analyses of GPS data [1-6], which is generated by devices pre-installed on moving objects and transmitted every few seconds. However, in many occasions, many vehicles may not pre-install GPS devices or just turn it off. For example, vicious trackers of taxies or bank cash carriers will not turn their GPS devices on. Hence, in this paper, we introduce a special kind of streaming traffic datasets, i.e., Automatic Number Plate Recognition (ANPR) data and explore a different way to discover traveling companions on it.

Today, in many cities in China, Traffic Enforcement Cameras (TECs) have been installed in most road crossings. These cameras continuously take pictures of passing vehicles with approximate one second interval. Vehicle information, e.g., plate number and passing time, is automatically recognized and transmitted to a central data center of traffic management department in form of data stream. Compared with GPS data, the ANPR dataset is very suitable for the round-the-clock and wide-range monitoring of vehicles without requiring pre-installing or turning on any extra devices.

The paper develops a framework to discover companion vehicles in an instant manner based on ANPR data. Compared to existing approaches to discover traveling companions [2-6], our main contribution is to instantly detect suspicious companion vehicles with their probability values when they pass through TEC. To do this, we propose a data structure, called Moment Companion, to record information of companion vehicles in a given moment for a vehicle. Dynamically generated Moment Companions are connected to form an evolving graph, which records historical information about all companions for each vehicle at each moment. Based on this graph, through computing the occurrence frequency of each companion vehicle, we can compute the corresponding probability and order suspicious companion vehicles. Experiments show our approach can achieve low latency when facing large-scale ANPR datasets.

II. PROBLEM DEFINITION AND ANALYSES

Traffic enforcement cameras are spread over road crossings. Each camera is a stream data source, which continuously generates ANPR data records. The generation rate depends on the frequency of the vehicles passing through it. During peak hours, the generation rate can achieve one record per second. Each ANPR record only contains information about one vehicle. Formal definition of ANPR data record is presented below.

Definition 1 (ANPR Data Record): an ANPR data record r is a three tuple denoted as r = (c, p, t), where c is a camera ID, p is a vehicle plate number and t is a timestamp.

An ANPR data record r = (c, p, t) means a vehicle with the plate number p passes through a camera c at time t. The following example shows some real ANPR data records:

(CAM10015122, JingMS7416, 2013-1-1 09:36:00),
(CAM09712112, HuQY1924, 2013-1-1 09:36:00),
(CAM04612111, JingQE4438, 2013-1-1 09:36:00).

Figure 1 illustrates the goal of our framework. We want to instantly identify the companions when a given vehicle passes through a camera. In each output, the companion vehicles are ordered according to the probability of companion relationship between them and the given vehicle. In Figure 1, when vehicle v1 passes through camera C1 at time T1, companion vehicles of v1 compose an ordered list: <v2: 67%, v4: 67%, v5: 33%>;

978-1-5090-5437-4/16 $31.00 © 2016 IEEE
DOI 10.1109/WISA.2016.27

then when v1 arrives at camera C2 in T2, the companion vehicles of v1 are changed to be v2, v5, v6. Corresponding output is <v2:75%, v4:50%, v5:50%, v6:25%>; then when v1 finally reaches camera C3 at T3, the companion vehicles of v1 are v2, v5, v7, and corresponding output is <v2:80%,, v5:60%, v4:40%, v7:20%, v6:20%>.

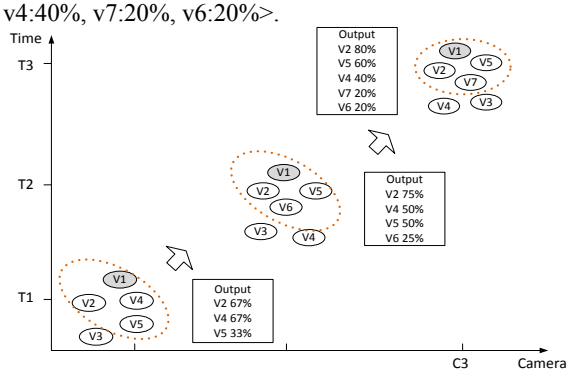


Figure 1. Instant Discovery of Travelling Companion

We summarize several requirements of discovering companion vehicles from above scenario: (1) Features of ANPR Data. The companions are vehicles that travel together, i.e., corresponding companion vehicles have correlations both in time and space. Note that the locations of the cameras are fixed. It means, vehicles passing through same camera closely can be regarded as companions. Hence, in this paper, we take the time interval of passing through a camera as the way to measure the correlation of two vehicles. (2) Volume and Velocity of ANPR Data. To instantly identify companion vehicles on huge ANPR data is the main challenge in this paper. For a big city like Beijing, TEC number may exceed 5000. During peak hours, the picture shooting frequencies can reach one picture per second. If there are 4 peak hours a day, the recognized ANPR data records can achieve 144 million records. To ensure the instantaneity, we design a parallel algorithm to process such large-scale dataset.

In this paper, we propose a framework to identify discover companion vehicles in an instant manner based on streaming ANPR datasets. In order to improve performance, we model ANPR data partition problem as a Balanced Number Partition Problem [7, 8] and propose a partition strategy called Balanced Partition Strategy to handle partition skew problem. Experiments prove that our approaches have short delay time and can quickly discover companion vehicles after the streaming ANPR data is received.

III. Instant Discovery of Companion Vehicle

3.1 *Discovery Framework*

Each camera can be regarded as a stream data source, which continuously generates ANPR data streams. The cloud data center accommodates multiple server machines to provide enough computation resources for the execution of our parallel algorithm.

The moment companion is the core data structure in our solution. Like a snapshot, it records companions of a given vehicle in a given moment when passing through a specific camera. To define a moment companion, we employ the concepts in this study.

Definition 2 (Moment Companion): Given a vehicle v and a camera c, let pt(v, c) be the moment that vehicle v passing through camera c, let Δt be the time threshold, then the moment companion of v at the moment passing through c can be defined as:

$$\mathrm{MC}(v, c, \mathrm{pt}(v, c)) = \{v' | \mathrm{pt}(v, c) - \Delta t \le \mathrm{pt}(v', c) \le \mathrm{pt}(v, c)\}$$

According to Definition 2, moment companion should record all vehicles information that accompany with the given vehicle in a given period. Time threshold Δt is a key factor when discovering moment companions. To compute moment companion of a vehicle v at a given moment t, we will collect information about vehicles that pass through the corresponding camera c in a time period [t-Δt, t]. The vehicles in this interval are regarded as the companion vehicles of the vehicle v.

To handle the large-scale and dynamic characteristics of ANPR data, our algorithm is designed and implemented on the Spark Streaming framework1. It is an extension of the core Spark API and enables scalable, high-throughput, fault-tolerant stream processing of live data streams. Resilient Distributed Dataset (RDD) is a core abstraction of Spark. It is a collection of elements partitioned across the nodes of the cluster that can be operated on in parallel. When designing the algorithm, we use two types of operations provided by Spark: transformations and actions. Transformations provide different ways to create a new dataset from an existing one. Actions return a value to the driver program after running a computation on the dataset. Details of Spark and Spark Streaming framework can be found at [9].

3.2 *Discovery of Moment Companion*

Figure 2 shows the process of discovering moment companions. Our framework contains a master node as well as several worker nodes. The master node is in charge of starting a computation job and assigning it to an idle worker node. A sliding window is built on the master node, which can cache incoming streaming ANPR data records for later processing. The master node also partitions cached ANPR dataset into blocks and dispatches them to worker nodes. Once a data block is received, the worker node will start to compute moment companions. The results are stored at the Hadoop distributed file system (HDFS) across the master and worker nodes.

Partition skew is the main challenge when we design partition strategies [10]. Partition skew occurs on parallel implementations when the workload is not balanced between nodes. To avoid partition skew problem, a challenge is to find an effective strategy to partition ANPR dataset in the sliding window into blocks. Our goal is to let the blocks have similar size and ANPR data records from the same camera should be put into the same block as much as possible. It is because, according to Definition 2, underlying companion vehicles only appear under the same camera. If data records from the same camera are partitioned into different blocks, we have to design complicated merge strategy to judge whether two discovered moment companions from different worker nodes should be combined later. It is also costly and brings lots of unnecessary burdens to the system. To deal with the above data partition challenge, we propose a balanced partition strategy and

[1] Spark Streaming framework, http://spark.apache.org/docs/latest/ streaming-programming-guide.html

transform it into a classical Balanced Number Partition Problem [7] to solve it.

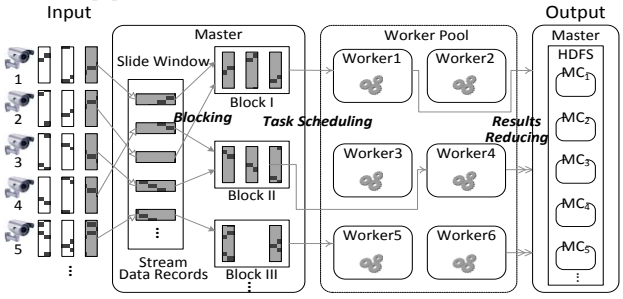


Figure 2. The Process of Discovering Moment Companions

Definition 3 (Balanced Partition Strategy): Given an ANPR dataset S, a balanced partition strategy bp is a partition function $bp(S) = \{S_1, S_2, \dots, S_k\}$, s.t.

(1) $S = \cup_{i=1..k} S_i$;

(2) $(S_i \cap S_j = \phi)\ i, j = 1 \dots k \wedge i \neq j$;

(3) given two data records r = (c, p, t) and $r' = (c', p', r')$, if $r.c == r'.c'$, then $\exists S_i (r \in S_i \wedge r' \in S_i)$;

(4) $\min(\sum_{i,j=1..k} |S_i| - |S_j|)$, where $|S_i|$ is the size of S_i.

To apply the Balanced Partition Strategy in Definition 3, we transform it into a classical Balanced Number Partition Problem (see in Definition 4). This problem is about how to divide an integer list L into k sublists and make sublist size as similar to each other as possible. Definition 4 uses definition in [7, 8] for reference.

Definition 4 (Balanced Number Partition Problem): Given a list L= {l1, l2,…, ln} of positive integers, let sum(L) be the sum of positive integers in L, partition L into k sublists L1,L2,…,Lk, s.t. max{sum(L1),…, sum(Lk)} is minimized.

To transform the problem, we first order ANPR data records based on their camera IDs. Based on camera information, an ANPR dataset can be roughly divided into several blocks. Each block contains ANPR data records from same camera and data records from different blocks are from different cameras. Then, the size of each block will form an integer list. Hence, the above strategy can be transformed into a Balanced Number Partition via Definition 4.

The Balanced Number Partition Problem is a NP-hard problem [7, 8]. However, an approximate approach named BLDM is given with time complexity O(nlogn) to solve it [7].

BLDM performs balanced k-way partitioning as follows. First, L is padded with zero-value numbers, so that $n = |L| = bk$ for some integer b. The numbers in L are then sorted in descending order. The sorted sequence, denoted by $(v_1, v_2, \dots, v_n)$, is split into b disjoint k-tuples $p_i = (v_{(i-1)k+1}, v_{(i-1)k+2}, \dots, v_{ik})$, for $1 \leq i \leq b$. The spread in each p_i is $\delta(p_i) = v_{(i-1)k+1} - v_{ik}$. Next, BLDM repeatedly replaces the two k-tuples p_i and p_j with the largest spreads with a new k-tuple p ; p is the result of folding p_i with p_j, by adding the first (largest) value in p_i with the last (smallest) value in p_j , the second (largest) value in p_i with the second last (smallest) in p_j , and so on. This process continues until single k-tuple remains. Each of the k elements in this final tuple corresponds to one sublist of the produced partitioning

Based on the above-mentioned partition strategy, we propose the corresponding moment companion discovery algorithm. It is implemented on top of the Spark Streaming framework.

Algorithm I. Discovery of Moment Companions

Function: DMC
Input: Δt be the time threshold, S be the ANPR dataset contains data records in the sliding window;
Output: a set of records of companion vehicles of v;

```
1.  window().foreach()
2.    Q←RDD.collect()
3.    partition Q by bp strategy based on BLDM algorithm
4.    Initial a list R, outputSet
5.    for each block Block
6.      Initial a JavaPairRDD blockRDD←parallelize(Block)
7.      for each record ri=(vi,ci,ti) in Block
8.        if Block contains enough data of ri and never discover moment
        companion of ri before
9.          outputSet .add(blockRDD.filter().collect())
10.         R.add(ri)
11.       if end
12.     for end
13.   for end
14.   Q.removeAll(R)
15. window().foreach() end
16. Initial a JavaRDD outputRDD←parallelize(outputSet)
17. outputRDD.saveAsTextFiles
```

Algorithm I shows the process of discovering moment companions. Algorithm I accepts input stream data by sliding window and caches them in Q (Lines 1-2). Then it partitions data in Q by function BLDM (Line 4). Next it computes moment companions in parallel on different worker nodes (Lines 6-12). Then unnecessary data is removed from Q on master node (Line 14). At last computing results are saved into HDFS (Lines 16-17).

3.3 *Discovery of Traveling Companion*

With Algorithm I, moment companion can be generated every time a vehicle passes through a camera. We can link these moment companions to form a graph, called as MC Linkage Graph (MCLG). Figure 3 shows a graph example. A node is a moment companion and is represented by a camera ID, a timestamp and a set of plate numbers with timestamp. Each node records the information of the vehicles passing through the same camera in a given time interval.

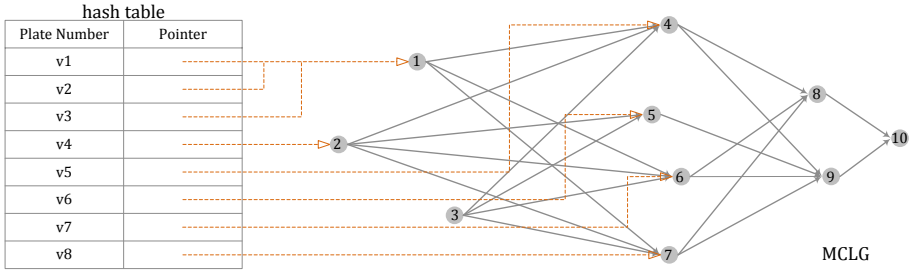
As Figure 3 shows, each node records the information of a moment companion at a time point. A directed edge is generated, if and only if there is a vehicle v satisfying that $v \in node1, v \in node4, node1.v.t > node4.v.t$, and node4.v.t is the most closed to node1.v.t in the MCLG. MCLG dynamically evolves when more moment companions are computed. Once a new moment companion is found, a new node of MCLG is constructed and added into the graph automatically.

Definition 5 (MC Linkage Graph): a MC Linkage Graph MCLG = (V, E), where V is a set of moment companions and E is a set of directed edges:

$\forall e \in E, e = (x, y)$ if $x, y \in V, \exists v, v \in x \wedge v \in y, x.v.t > y.v.t, \nexists z \in V, s.t., x.v.t > z.v.t > y.v.t$.

Based on the definition, we can easily get that there is not a two way edges between any two nodes in a MCLG. Figure 3 shows an example of different types of nodes in a MCLG. Via the MCLG, we can backtrack a given vehicle and find its historical moment companions. However, only recent historical

moment companions are useful for the computation of companion probability. Hence, we let users decide the maximal period to search historical moment companions and take it as the base to compute the companion probability. To do this, we set a time threshold $\delta_t$to search historical moment companions from the MCLG graph. In other words, let the latest timestamp of a given vehicle v is captured be t, then historical moment companions of v in [t -δ_t, t] are searched and put into a set. Then, based on a frequency theory, we can compute companion probabilities.



content on nodes	
1:camID=c_5,t=9:59:59,MC={(v_1,9:59:59),(v_2,9:59:58),(v_3,9:59:57)}	2:camID=c_5,t=9:59:58,MC={(v_2,9:59:58),(v_3,9:59:57),(v_4,9:59:56)}
3:camID=c_5,t=9:59:57,MC={(v_3,9:59:57),(v_4,9:59:56),(v_7,9:59:55)}	4:camID=c_4,t=9:54:30,MC={(v_1,9:54:30),(v_4,9:54:29),(v_5,9:54:28)}
5:camID=c_4,t=9:54:29,MC={(v_4,9:54:29),(v_5,9:54:28),(v_6,9:54:27)}	6:camID=c3,t=9:54:30,MC={(v_2,9:54:30),(v_3,9:54:29),(v_7,9:54:28)}
7:camID=c_3,t=9:54:29,MC={(v_3,9:54:29),(v_7,9:54:28),(v_8,9:54:27)}	8:camID=c_2,t=9:50:25,MC={(v_1,9:50:25),(v_2,9:50:24),(v_3,9:50:23)}
9:camID=c_2,t=9:50:25,MC={(v_4,9:50:20),(v_6,9:50:19),(v_7,9:50:18)}	10:camID=c_1,t=9:47:10,MC={(v_2,9:47:10),(v_3,9:47:09),(v_7,9:47:08)}

Figure 3. An Example of MC Linkage Graph, $\Delta t = 2s$.

To improve the performance of traversing a MCLG, we design a hash table. For each vehicle captured by a camera, we set its plate number as a key, set a pointer as its corresponding value. The pointer only points to the node which saves the information of latest capture for this vehicle. It is to say that given a vehicle v, its pointer in the hash table points to a node n in a MCLG. Then there is not a node m satisfying that m ≠ n, and m.v.t>n.v.t. Figure 3 shows an example of the hash table.

Given a vehicle v we can directly locate the node in the MCLG by the hash table. Then we get all nodes which are linked to this node and contain the given vehicle. For these discovered nodes, search their neighbors in the same manner till all nodes contains vehicle v are founded.

Taking Figure 3 as example, given a vehicle v_2, we get the node 1 according to the hash table at the top left corner of this figure. In the MCLG, node 6 is the only one which is linked to node 1 and contains v_2. Search all neighbors of node 6, node 2 and node 8 are selected out (node 1 is already there). Search neighbors of node 8 and we get node 10.

Based on probability theory, we can compute the probability of occurrence of an event based on its occurrence frequency [11]. Frequency is the number of occurrences of a repeating event per unit time. Definition 6 below defines how to compute the probability of an event based on its frequency.

Definition 6 (Probability of Event E_i Occurrence): Given a sample space of events Ω and an event $E_i \in \Omega$,, the probability of occurrence of event E_i is defined as follows:

$$P(E_i) = N_{E_i}/N_\Omega$$

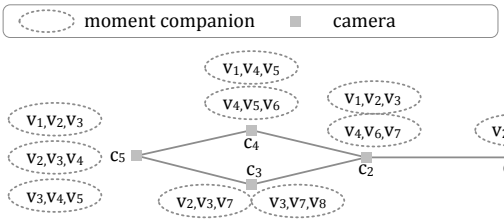
where N_Ω is the occurrence times of all events in Ω and N_{E_i} is the occurrence times of E_i.

In our case, for a given vehicle v_1 and an arbitrary vehicle v_2, we can define $\Omega = \{E_1, E_2\}$ where E_1 represents the event that v_1 and v_2 simultaneously occurs in a moment companion and E_2 represents the opposite. Consequently, we can give the concept of probability of v_2 being a companion vehicle of v_1.

Definition 7 (Companion Probability): Let v1, v2 be two vehicles, let δ_t be the predefined time threshold, curTime be the latest captured moment of v1, let the corresponding historical moment companions set of v1 be $S_{MC}(v_1, \delta_t, curTime) = \{MC_1, MC_2, \dots, MC_n\}$ during time period [curTime - δ_t, curTime], then the probability of v2 being a companion vehicle of v1 can be defined as:

$$P(E_1) = N_{E_1}/|S_{MC}|$$

where N_{E_1} is the times of v_2 being a companion vehicle of v_1 in S_{MC} and $|S_{MC}|$ is the size of S_{MC}.



9:47:10	v_3:100%;v_7:100%
9:50:25	v_3:100%;v_1:50%;v_7:50%
9:54:30	v_3:100%;v_7:66%;v_1:33%
9:59:58	v_3:100%;v_4:33%;v_7:33%;v_1:33%
9:59:59	v_3:100%;v_1:66%;v_7:33%;v_4:33%

Figure 4. Example: How the Probability of Each Companion Vehicle Changes

Figure 4 illustrates the changes of companion probability. The left side is a road map from Figure 3. Each oval is a moment companion. Each grey square is a camera. Let δ_t be 10min, figure 4 shows the probabilities of all companion vehicles of v_2 from moment 09:47:10 to moment 09:59:59. Here, the historical moment companion node set of v_2 at

09:47:10 is $S_{MC}(v_2, 10min, 9:47:10) = \{node10\}$; correspondingly, $S_{MC}(v_2, 10min, 9:50:25) = \{node8, node10\}$, $S_{MC}(v_2, 10min, 9:54:30) = \{node6, node8, node10\}$, $S_{MC}(v_2, 10min, 9:59:58) = \{node2, node6, node8\}$, $S_{MC}(v_2, 10min, 9:59:59) = \{node1, node6, node8\}$.

Furthermore, we can get $S_{MC}(v_2, 10min, 9:59:59) = \{node1, node6, node8\}$. Taking v_3 as an example, since $|S_{MC}(v_2, 10min, 9:59:59)| = 3$, $P(v_3) = \frac{3}{3} = 100\%$.

IV. EXPERIMENT

4.1 *Experiment Setup*

The paper focuses on detecting the companion vehicles based on ANPR data. It means the results should be generated as soon as possible while receiving corresponding ANPR data records. Therefore, we design the following two indexes to measure time delays.

Definition 8 (Processing Time): For an ANPR data record r, let t_s and t_e represent its arrival time and the end time of processing, the processing time $t_{pt}$is $t_{pt} = t_e - t_s$.

Definition 9 (Total Processing Time): For an ANPR dataset R={r1, r2, …, rm}, the total processing time is defined as:

$$t_{tpt} = \sum t_{pt}(r_i) \quad i = 1..m$$

The experiments are done on real ANPR dataset in Beijing, China. The data records are the recognized vehicle plate information from 2012-12-06 17:57:53 to 2013-01-04 22:03:54. To verify the relationship between data scale and process time, we extract ten thousands to millions of records from this dataset and run Algorithm I with them on 1, 3, 5 nodes respectively. Besides, we set Δt = 2 min in Algorithm I.

Our experiments are finished on a cluster consisted of 5 nodes. All nodes are running in virtual machines with CentOS release 6.4 and java 1.70. Detailed configuration of our cluster is shown in Table I. We used Spark version 1.1.0 and Hadoop version 2.3.0. And all results are stored in HDFS file system.

TABLE I. CONFIGURATION OF OUR CLUSTER

Role in cluster	*CPU*	*Core No.*	*Memory*
master	Intel Xeon E312xx	6	6G
worker1	Intel Xeon E312xx	6	6G
worker2	Intel Xeon E312xx	3	3G
worker3	Intel Xeon E312xx	3	3G
worker4	Intel Xeon E312xx	3	3G

4.2 *Experiment Result*

4.2.1 Data Arrive Rate and Processing Time

In this part, we mainly analyze how data arrival rate impacts processing times. Figure 5 shows data arrival rate of our 1,160,921 ANPR data records. From this figure, we can find that maximum arrival rate can achieve 140 records per second and minimum arrival rate is about 40 records per second.

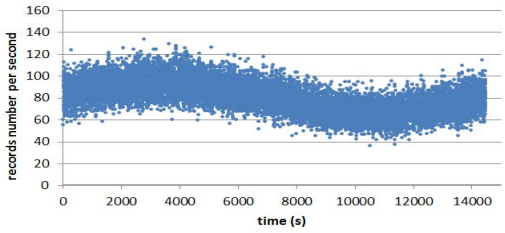


Figure 5. Data Arrival Rate of 1,160,921 ANPR Data Records

Figure 6 presents Processing Time of Algorithm I with 1, 3 and 5 worker nodes respectively where Δt=2min. Here we choose master node under 1 node case; master, worker2 and worker3 under 3 nodes case. Under 1 node case, master node acts as master and worker both. As Figure 6(a) shows, Processing Time with one node is a parabola. It first increases dramatically when input data arrives. Then the parabola decreases dramatically. Our analyses show the main reason is one node cannot effectively handle the large-scale input data at the beginning. Lots of time delays emerge until all data records are arrived. Figure 6(b) shows Processing Times with 3 worker nodes. We find the wave is quite similar with Figure 6, yet the data can be better handled with 3 nodes. Figure 6(c) shows Processing Times with 5 worker nodes. The wave in this figure is more gradual than it with 3 nodes. From this figure, we can draw the conclusion that more worker nodes can better handle the high arrival rates of stream data.

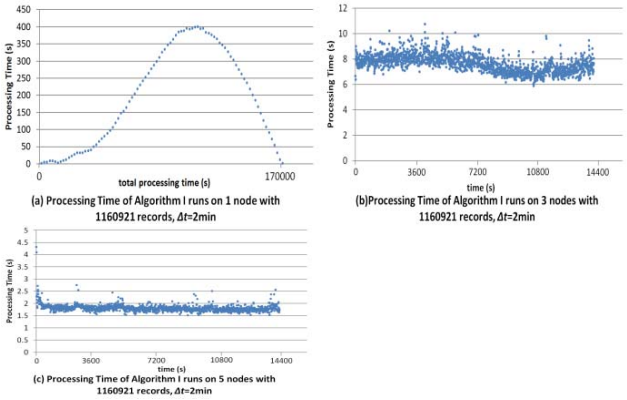


Figure 6. Processing Time of Algorithm I with 1,160,921 records, Δt=2min

4.2.2 Data Scale and Total Processing Time

Table II and Figure 7 shows the relationship between the data scale and the corresponding Total Processing Time. We can draw the conclusion that the better performance can be achieved when more nodes are used. The curve line of one node increases sharply when processing larger scale data. It means that one node cannot effectively handle our problem. In addition, with parallel computation on five nodes, the Total Processing Time is around 25% of the value with three nodes.

TABLE II. PROCESSING TIME WITH DIFFERENT DATA SCALES

Data record no.	*1 node*	*3 nodes*	*5 nodes*
10,269	114.23s	82.65s	22.18s
104,409	1265.15s	825.30s	198.90s
1,160,921	171677.85s	9055.06s	2728.51 s

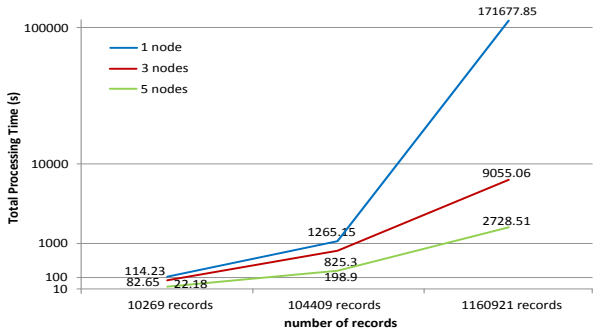


Figure 7. Total Processing Time

V. Related Work

Many researchers have put their interests on traveling companion study. Most of the studies are designed to work on static datasets on 2D Euclidean space. [1] proposed a definition flock as a group of moving objects moving in a disc of a fixed size for k consecutive timestamps. [2] proposed convoy, an extension of flock, where spatial clustering is based on density. [3] proposed swarm which enables the discovery of interesting moving object clusters with relaxed temporal constraint. [12] proposed a new type of patterns, platoon patterns, that describes object clusters that stay together for time segments, each with some minimum consecutive duration of time. More recently, researchers begin to pay attention to large scale trajectory. [6] developed a set of techniques to improve the performance of discovering gathering patterns over static large-scale trajectory databases. [13] proposed a gathering retrieving algorithm to retrieve gathering pattern by searching a spatio-temporal graph composed of the moving object clusters. [14] tried to leverage the MapReduce framework to achieve higher spatial data processing efficiency. It also proposed a partition strategy to avoid spatial relationships missing. Both [6] and [13] focused on algorithm design instead of parallelization.

The above studies cannot effectively handle stream data. However, in many traffic applications, data sources are in the form of stream. Hence, in recent years, more and more studies began to process streaming traffic data. [4] proposed a framework to discover travelling companion among streaming trajectories and output results in an incremental manner. [15] [16] studied on a density-based clustering algorithm for trajectory data stream and tried to discover trajectory clusters in real time. All these relative work aimed at processing GPS stream data and made progress. They provide a foundation for our study. But they did not pay attention to algorithm parallelization. In this paper, we propose a framework to identify traveling companions in near real time on a special kind of streaming traffic datasets called ANPR dataset. We parallel our approach and implement it on top of Spark Streaming computing framework.

VI. Conclusion

In this paper, we propose a framework and corresponding algorithms to instantly discover companion vehicles. Our framework takes streaming ANPR data records as input and processes data in parallel. The main contribution is that the framework can instantly detect suspicious companion vehicles with their probabilities when they pass through monitoring cameras. Experiment results show that our method can output companion vehicles in near real-time. In the future, we plan to take more traffic data like GPS data into account. It can extend the ability of our framework and wide its applications.

Acknowledgment

The research work is partially supported by the following projects: Key Program of Beijing Municipal Natural Science Foundation “Theory and Key Technologies of Data Space Towards Large Scale Stream Data Processing” (No. 4131001); Training Plan of Top Young Talent in North China University of Technology, “An Incremental Approach to Instant Discovery of Data Correlations Among Multi-Source and Large-scale Sensor Data”.

References

[1] Gudmundsson, J. M. V. Kreveld, Computing longest duration flocks in trajectory data. ACM GIS, 2006.

[2] H. Jeung, M. L. Yiu, X. ZHOU, C. S. Jensen, H. T. Shen, Discovery of convoys in trajectory databases,VLDB, 2008.

[3] Z. Li, Z., B. Ding, J. Han, R. Kays, Swarm: Mining relaxed temporal moving object clusters accurate discovery of valid convoys from moving object trajectories, VLDB, 2010.

[4] L. A. Tang, Y. Zheng, J. Yuan, J. Han, A. Leung, C. C. Hung, W. C. Peng, On discovery of traveling companions from streaming trajectories, in: Data Engineering (ICDE), 2012 IEEE 28th International Conference on, IEEE, 2012, pp. 186-197.

[5] L. A. Tang, Y. Zheng, J. Yuan, J. Han, A. Leung, C. C. Hung, W. C. Peng, P. T. La, A Framework of Traveling Companion Discovery on Trajectory Data Streams, ACM Trans. Intell. Syst. Technol., 2014, 5, pp. 1-34.

[6] K. Zheng, Y. Zheng, N. J. Yuan, S. Shang. On discovery of gathering patterns from trajectories, Data Engineering (ICDE), 2013 IEEE 29th International Conference on. IEEE, 2013, pp. 242-253.

[7] W Michiels, J Korst, E Aarts. Performance Ratios for the Differencing Method Applied to the Balanced Number Partitioning Problem[M] STACS 2003. Springer Berlin Heidelberg, 2003:583-595.

[8] R E Korf. Multi-Way Number Partitioning.[C]. Proceedings of the 21st international jont conference on Artifical intelligence. Morgan Kaufmann Publishers Inc., 2009:538-543.

[9] Spark, http://spark.apache.org/ last retrieved at 2015/1/10.

[10] Walton C B, Dale A G, Jenevein R M. A taxonomy and performance model of data skew effects in parallel joins[C] Proc. of Int'l Conference on Very Large Data Bases, 17th, 1991. 1991:537-548.

[11] L. Cosmides, J. Tooby, Are humans good intuitive statisticians after all? Rethinking some conclusions from the literature on judgment under uncertainty. cognition, 1996, 58(1),pp. 1-73.

[12] Y X Li, J Bailey, L Kulik. Efficient Mining of Platoon Patterns in Trajectory Databases ☆[J]. Data & Knowledge Engineering, 2015, 100(PA):167-187.

[13] J. Zhang, J. Li, S. Wang, Z. Liu, Q. Yuan, F. Yang. On Retrieving Moving Objects Gathering Patterns from Trajectory Data via Spatio-temporal Graph, Big Data (BigData Congress), 2014 IEEE International Congress on. IEEE, 2014, pp. 390-397.

[14] J. S. Yoo, D. Boulware, D. Kimmey. A Parallel Spatial Co-location Mining Algorithm Based on MapReduce, Big Data (BigData Congress), 2014 IEEE International Congress on. IEEE, 2014,pp. 25-31.

[15] Y. Yu, Q. Wang, X. Wang, Continuous clustering trajectory stream of moving objects, Communications, 2013, 10(9), pp. 120-129.

[16] Y. Yu, Q. Wang, X. Wang, H. Wang, J. He. Online clustering for trajectory data stream of moving objects, Computer Science and Information Systems, 2013, 10(3), pp. 1293-1317.